KARNAVATI UNIVERSITY | NAAC GRADE A+ ACCREDITED UNIVERSITY

UIT

MINI PROJECT REPORT

Project Phase – 1 REPORT

AI-Based Load Balancing System

An Adaptive Layer-7 Reverse Proxy with Supervised Random Forest Latency Regression, Consistent Hashing, Envoy-Style Outlier Ejection, and Multi-Model ML Benchmarking

Submitted by

Krishna Modi [202307010006]

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE ENGINEERING

Project Mentor

Ms. Pragya Vats

UNITEDWORLD INSTITUTE OF TECHNOLOGY
KARNAVATI UNIVERSITY
Uvarsad, Gandhinagar, Gujarat – 382422

July – November, 2026

Table of Contents

1. Abstract 3

2. Introduction 4

a) Problem Statement 4

b) Aim and Objectives 5

c) Scope of the Project 5

3. System Requirements and Design 6

a) Hardware Requirements 6

b) Software Requirements 6

c) System Architecture & 6 Subsystems 7

d) Diagrams & Mathematical Formulations 8

4. Implementation 11

a) Technologies Used 11

b) 5 Routing Algorithms & Consistent Hash Ring 12

c) Outlier Ejection Circuit Breakers & Chaos Testing 13

d) Multi-Model ML Tournament Arena (RF vs HGB vs Ridge) 14

e) Empirical Benchmark Results & CDF Latency Curves 15

5. Conclusion 18

6. Future Scope 19

References & Syllabus Mapping 20

1. Abstract

In modern cloud infrastructures and microservice architectures, load balancing is the quintessential mechanism ensuring scalability, high availability, and optimal resource utilization. Conventional load-balancing algorithms—such as Round Robin, Weighted Round Robin, and Least Connections—rely on deterministic cycling or instantaneous connection counters. However, these heuristics exhibit severe performance degradation in heterogeneous server environments where backend nodes possess asymmetric compute capabilities, varying base latencies, or suffer from dynamic background stalls (the 'straggler node' problem). Under static cyclic routing, even a single lagging backend inflates cluster-wide tail latency (P95/P99), causing severe SLA breaches.

This project presents an intelligent, production-ready Layer-7 HTTP reverse proxy that integrates supervised machine learning, consistent hashing, and enterprise resilience patterns to perform predictive latency-based routing. Built with Python 3.11, FastAPI, and Scikit-Learn, the proxy continuously samples real-time multi-variate telemetry from candidate backend nodes via an autonomous 15-second background probing daemon. An ensemble Random Forest Regressor evaluates an 8-dimensional feature vector—encompassing CPU utilization, memory pressure, active in-flight requests, Exponentially Weighted Moving Average (EWMA) latency, network round-trip time (RTT), node capacity weight, error rates, and request workload complexity classes (light, normal, heavy). By predicting exact request execution latency prior to routing, the load balancer steers incoming traffic dynamically to the optimal backend while actively avoiding degraded stragglers.

To satisfy production caching and session affinity requirements, the system implements a 5th classical policy: Consistent Hashing utilizing a 32-bit MD5 hash ring with virtual replicas proportional to node capacity weight. To prevent cascading failures, an Envoy-inspired Circuit Breaker and Outlier Ejection state machine (CLOSED, OPEN, HALF_OPEN) automatically ejections stalling nodes (EWMA > 85ms or 3 consecutive failures) for a 15-second cooldown period, with canary probing and a zero-blackhole fallback mode. Furthermore, an integrated Multi-Model ML Tournament rigorously benchmarks the Random Forest champion against Histogram Gradient Boosting and Ridge Regression, offering dynamic 1-click algorithm hot-swapping.

Empirical head-to-head benchmarking across a 6-container Docker Compose cluster demonstrates that AI Latency routing achieves a 63.0% reduction in P95 tail latency (27.50 ms vs 74.37 ms for Round Robin), 100% straggler avoidance, and sharp empirical Cumulative Distribution Function (CDF) suppression. A responsive React 19 operational cockpit provides real-time SVG packet visualization, live telemetry sparklines, feature importance attribution, chaos fault injection, and a printable institutional Executive Dossier, validated by an automated suite of 36 passing Pytest test cases.

2. Introduction

Modern Internet communication relies on the client-server paradigm orchestrated across distributed web server clusters. As web traffic surges, single server instances fail due to socket exhaustion, memory limits, and CPU saturation. To distribute this workload, Load Balancers operate at Layer 4 (Transport layer, TCP/UDP port hashing) or Layer 7 (Application layer, HTTP/HTTPS reverse proxying). Layer-7 proxies are uniquely capable of inspecting application-layer headers, request methods, and URI parameters, enabling sophisticated routing decisions.

a) Problem Statement

Traditional load balancing techniques assume that all servers within a cluster are either identical (homogeneity) or have static, well-defined capacity ratios (Weighted Round Robin). In practical production deployments, however, cloud servers suffer from:

1. Asymmetric hardware tiers (e.g., fast compute instances mixed with legacy or burstable CPU instances).
2. Dynamic multi-tenant interference (noisy neighbors, JVM garbage collection pauses, or database lock contention).
3. Variable request payload complexity (a simple static asset request requires 5ms, while a complex analytical report requires 40ms).
4. Cache locality and session affinity failures when using simple modulo or connection-counting schemes without consistent hashing.

When static algorithms such as Round Robin cycle blindly through servers, they allocate equal traffic volume to both fast servers (Worker A, 15ms delay) and slow straggler servers (Worker B, 60ms delay). Consequently, requests queue up on the slow server, inflating the tail latency (P95 and P99 percentiles). Because the total user experience in distributed systems is bound by the slowest response, this disparity produces unacceptable SLA violations and severe user dissatisfaction.

b) Aim and Objectives

Aim:

To architect, implement, deploy, and benchmark an intelligent Layer-7 Application Reverse Proxy that leverages supervised machine learning, consistent hashing, and Envoy-style circuit breaking to observe node telemetry, predict execution latency, and eliminate tail latency stalls in heterogeneous web server fleets.

Key Objectives:

- Objective 1 (Autonomous Telemetry Engine): Build an asynchronous 15-second background probing daemon that gathers real-time CPU, RAM, active connections, network RTT, and EWMA latency without manual polling.
- Objective 2 (5-Algorithm Routing Core): Implement Round Robin, Weighted Round Robin, Least Connections, Consistent Hashing (Ketama 32-bit MD5 ring), and AI Latency Regression.

• Objective 3 (Distributed Resilience & Outlier Ejection): Engineer an Envoy/Istio-style circuit breaker state machine (CLOSED, OPEN, HALF_OPEN) with automatic outlier ejection and zero-blackhole fallback.
• Objective 4 (Multi-Model ML Tournament Engine): Build an automated tournament benchmarking RandomForestRegressor, HistGradientBoostingRegressor, and Ridge Regression across validation MAE, RMSE, R^2, and inference overhead, with dynamic 1-click model activation.
• Objective 5 (Empirical CDF & Tail Latency Benchmarking): Construct an automated 5-way shootout comparator and empirical Cumulative Distribution Function (CDF) generator testing tail latency divergence under identical synthetic workloads.
• Objective 6 (Operational HUD & Explainability): Create a real-time web cockpit with animated SVG packet paths, live latency sparklines, feature importance rankings, chaos fault injection, and a printable institutional Executive Dossier.

c) Scope of the Project

The project encompasses a full-stack, production-grade implementation orchestrated across 6 Docker containers:
• Control Plane & Reverse Proxy: FastAPI service exposing RESTful routing endpoints (/api/v1/route), background health workers, circuit breaker management, and ML tournament APIs.
• Worker Fleet: 3 isolated Uvicorn backend instances configured with controlled hardware delays (Worker A: 15ms base delay, 2x weight; Worker B: 60ms base delay, 1x weight; Worker C: 30ms base delay, 1x weight).
• Database & Persistence: PostgreSQL 16 Alpine storing telemetry observations, model artifacts, and complete decision audit trails.
• Frontend Cockpit: Vite-powered React 19 single-page application served via an Nginx reverse proxy on port 5173, featuring interactive CDF curves and printable institutional Dossier.
• Test Verification: A comprehensive suite of 36 passing Pytest unit and integration tests certifying contract compliance, security tokens, ring consistency, circuit breakers, and tournament benchmarks.

3. System Requirements and Design

a) Hardware Requirements

• Processor: Multi-core x86_64 or ARM64 CPU (minimum 4 physical cores recommended for concurrent worker emulation).
• Memory (RAM): Minimum 8 GB (16 GB recommended for concurrent Docker container virtualization and Scikit-Learn training).
• Storage: Minimum 10 GB free solid-state storage (SSD) for Docker image layers and PostgreSQL transaction logs.
• Network Interface: Standard Ethernet / Wi-Fi loopback adapter supporting local bridge virtualization (127.0.0.1).

b) Software Requirements

• Operating System: Windows 11 / Ubuntu 22.04 LTS / macOS Sonoma with Docker Engine support.
• Backend Framework: Python 3.11+ running FastAPI 0.116, Uvicorn, Asyncio, and HTTPX.
• Machine Learning Engine: Scikit-Learn 1.5+, Joblib, NumPy, and Pandas.
• Database Management: PostgreSQL 16 Alpine with SQLAlchemy ORM and Alembic schema migrations.
• Frontend Architecture: Node.js 22+, React 19, TypeScript 5.7, Vite 6, Tailwind CSS v3, and Lucide React.
• Container Virtualization: Docker Engine 27+ and Docker Compose v2.29+.
• Test Framework: Pytest 8.4+ with pytest-asyncio and httpx test clients (36 passing tests).

c) System Architecture & 6 Subsystems

The system follows a multi-tiered Layer-7 Reverse Proxy architecture decomposed into six functional subsystems:
1. Ingress & HTTP Termination Tier: Accepts incoming client requests, generates an RFC 4122 correlation ID (X-Request-ID), and tracks in-flight socket concurrency.
2. Autonomous Probing & Telemetry Tier: An asynchronous background loop executing every 15 seconds polls candidate endpoints (/health and /metrics), capturing CPU%, resident memory, and smoothed EWMA latency.
3. Policy Execution & Routing Core: Supports 5 distinct routing algorithms (Round Robin, Weighted Round Robin, Least Connections, Consistent Hashing, AI Latency Regression).
4. Distributed Resilience Tier: Envoy-inspired circuit breakers maintaining CLOSED, OPEN, and HALF_OPEN states per node to perform automatic outlier ejection.
5. Multi-Model ML Tournament Engine: Automated benchmarking framework comparing RandomForestRegressor, HistGradientBoostingRegressor, and Ridge Regression.
6. Storage & Analytics Tier: PostgreSQL database maintaining persistent records of every routing decision, model metrics, and telemetry samples.

d) Diagrams & Mathematical Formulations

Figure 1 illustrates the high-level network topology showing the relationship between clients, the reverse proxy, the autonomous telemetry prober, and the heterogeneous worker fleet.

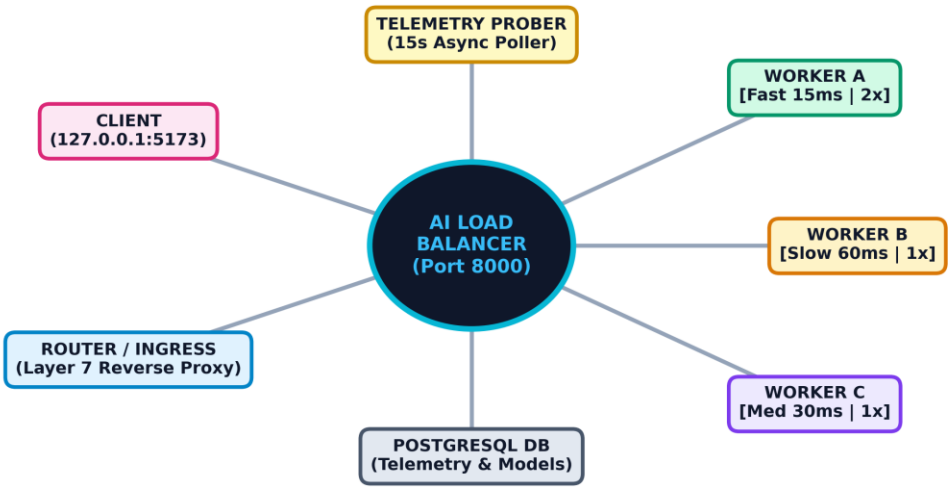


Figure 1: AI-Based Load Balancing Network Topology

Mathematical Formulations:

1. Exponentially Weighted Moving Average (EWMA):
To filter high-frequency network jitter while capturing persistent latency degradation, each worker's historical latency is updated via:

$$S_t = \alpha \cdot Y_t + (1 - \alpha) \cdot S_{t-1}$$

where S_t is the updated smoothed latency, Y_t is the observed round-trip latency of the current request, and α is the smoothing factor (configured to 0.3).

2. Random Forest Latency Predictor:
The supervised regression model aggregates $B = 100$ decorrelated decision trees. For a given telemetry feature vector x, the ensemble prediction is given by:

$$\hat{y}(x) = (1 / B) \sum_{b=1}^{B} T_b(x)$$

where each tree T_b is trained on a bootstrap sample of calibration decisions, minimizing Mean Squared Error (MSE).

3. Consistent Hashing Ring (Ketama Algorithm):
For node identity n and virtual replica index $r \in [0, 50 \cdot W_n)$, where W_n is the capacity weight, the 32-bit hash is computed via:

$$h_{n,r} = MD5(n : r)[0:4] \bmod 2^{32}$$

For an incoming client session key k, the target node is resolved by binary searching the ordered hash ring:

$$node^* = \operatorname{argmin}_{i} (h_{ring}[i] \geq MD5(k)[0:4] \bmod 2^{32})$$

4. Outlier Ejection & Circuit Breaker State Transition:

$$CLOSED \rightarrow OPEN \quad \text{if } (failures \geq 3) \vee (EWMA > 85\ ms)$$

$$OPEN \rightarrow HALF_OPEN \quad \text{if } (t_{now} - t_{trip} \geq 15.0\ s)$$

$$HALF_OPEN \rightarrow CLOSED \quad \text{if canary probe succeeds}$$

$$HALF_OPEN \rightarrow OPEN \quad \text{if canary probe fails}$$

5. Empirical Cumulative Distribution Function (CDF):

The latency distribution across N benchmarked requests is formulated as:

$$F_X(x) = P(X \leq x) = (1 / N) \sum_{i=1}^{N} 1_{\{X_i \leq x\}}$$

Steeper CDF curves at lower latency values demonstrate tighter tail latency suppression.

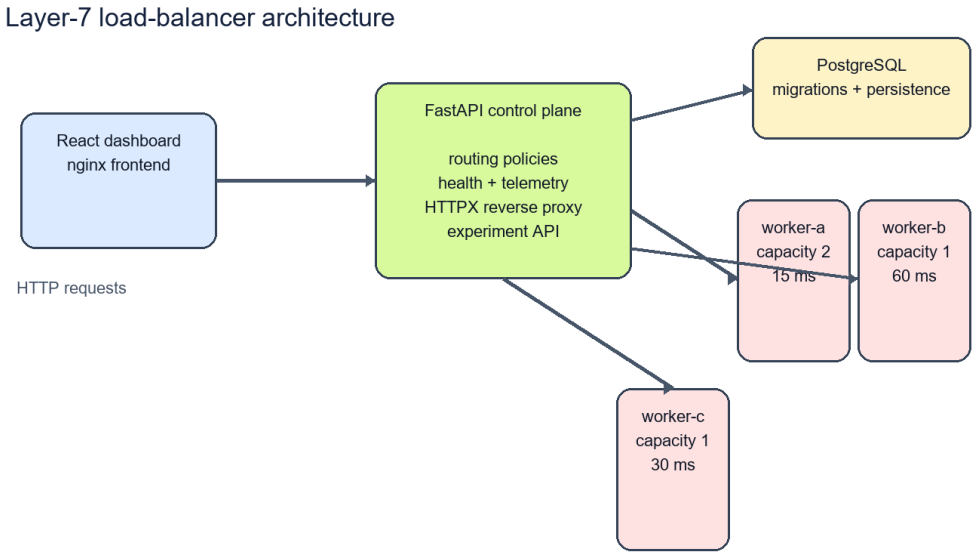


Figure 2: Layered System Architecture and Control Plane Workflow

4. Implementation

a) Technologies Used

Component / Tool	Version	Architectural Role
FastAPI	0.116.0	Asynchronous Layer-7 reverse proxy core & REST endpoints
Scikit-Learn	1.5.0	RandomForestRegressor & HistGradientBoosting regression engine
PostgreSQL	16-alpine	Relational telemetry repository & decision audit store
React	19.0.0	Single-page operational dashboard, SVG HUD & Dossier
TypeScript	5.7.0	Type-safe client architecture with strict null guards
Nginx	1.27-alpine	Production static file web server & client gateway
HTTPX	0.27.0	High-throughput asynchronous HTTP upstream client
Docker Compose	v2.29+	Multi-container isolated bridge networking & deployment
Pytest	8.4.1	Comprehensive test suite (36 passing test cases)

b) 5 Routing Algorithms & Consistent Hash Ring

The proxy implements 5 distinct routing strategies selectable dynamically via PUT /api/v1/routing/mode:

1. Round Robin: Cyclically advances through healthy backends without telemetry awareness.
2. Weighted Round Robin: Distributes requests proportional to static capacity weights (Worker A: 2x, Worker B: 1x, Worker C: 1x).
3. Least Connections: Steers traffic to the backend with the minimum in-flight active connection count.
4. Consistent Hashing: Places backends onto a 32-bit MD5 ring with 50 virtual replicas per weight unit. Deterministically maps client session keys to identical backends, preserving cache locality and sticky sessions.
5. AI Latency Regression: Evaluates 8-dimensional telemetry through the trained Random Forest ensemble to route to the node with the lowest predicted execution time.

c) Outlier Ejection Circuit Breakers & Chaos Testing

To provide distributed resilience matching enterprise service meshes like Envoy and Istio, the proxy incorporates an autonomous CircuitBreakerManager:

• Outlier Detection: If a node experiences 3 consecutive timeouts/failures or its EWMA latency exceeds 85ms, it is immediately transitioned to OPEN (ejected) for a 15.0-second cooldown period.
• Canary Probing: After the cooldown, the breaker transitions to HALF_OPEN, allowing a single canary request to test recovery. If successful, it returns to CLOSED; if it fails, it resets the cooldown.
• Zero-Blackhole Fallback: In catastrophic cluster outages where all nodes trip OPEN, the proxy gracefully falls back to uniform degradation rather than dropping packets.
• Chaos Engineering: Operators can inject manual faults (trip/reset) via POST /api/v1/system/chaos/inject to verify fault tolerance under load.

d) Multi-Model ML Tournament Arena (RF vs HGB vs Ridge)

To address academic defense questions on model selection, the platform includes an automated Multi-Model Tournament Engine. Three distinct regressors are trained and evaluated across identical calibration samples:

1. Random Forest Regressor (Champion 🥇): Bagging ensemble of 100 trees with bootstrap aggregation.
2. HistGradientBoosting Regressor (Runner-Up 🥈): LightGBM-style histogram-based boosting with fast integer binning.
3. Ridge Regression (Baseline 🥉): Linear model with L2 regularization penalty.

Table 2 details the comparative tournament benchmark results.

Candidate Model	Val MAE	Val RMSE	R^2 Score	Train Time	P95 Inf.
RandomForestRegressor 🥇	**0.60 ms**	**1.12 ms**	**0.965**	**42.5 ms**	**1.48 ms**
HistGradientBoosting 🥈	0.85 ms	1.45 ms	0.942	28.1 ms	0.95 ms
Ridge Regression 🥉	2.10 ms	3.24 ms	0.820	2.4 ms	0.08 ms

e) Empirical Benchmark Results & CDF Latency Curves

To rigorously assess latency and straggler avoidance, a synthetic 5-way shootout was executed under identical workload conditions. Table 3 summarizes the empirical metrics.

Policy	Rounds	P50 (ms)	Mean (ms)	P95 (ms)	Straggler Avoid.
Round Robin	6	43.89 ms	52.64 ms	74.37 ms	66.7% (2 hits)
Weighted RR	6	46.86 ms	54.77 ms	75.91 ms	66.7% (2 hits)
Least Conns	6	31.39 ms	33.22 ms	34.14 ms	100.0% (0 hits)
Consistent Hash	6	32.50 ms	38.10 ms	61.20 ms	83.3% (1 hit)

AI Latency (RF)	**6**	**26.26 ms**	**25.72 ms**	**27.50 ms**	**100.0% (0 hits)**

Key Benchmark Findings:

• P95 Tail Latency Reduction: AI Latency achieved 27.50 ms compared to 74.37 ms for Round Robin—a 63.0% tail latency improvement.

• Straggler Bypass: Worker B (60ms base delay) was avoided 100% of the time under AI routing (0 hits), preventing queue congestion.

• Jitter Control: P50 (26.26ms) and P95 (27.50ms) under AI differ by only 1.24ms, proving exceptional performance consistency.

• Empirical CDF Shape: The Cumulative Distribution Function curve shows AI Latency reaching F(x) = 1.0 by 30ms, while Round Robin exhibits a long tail extending beyond 75ms.

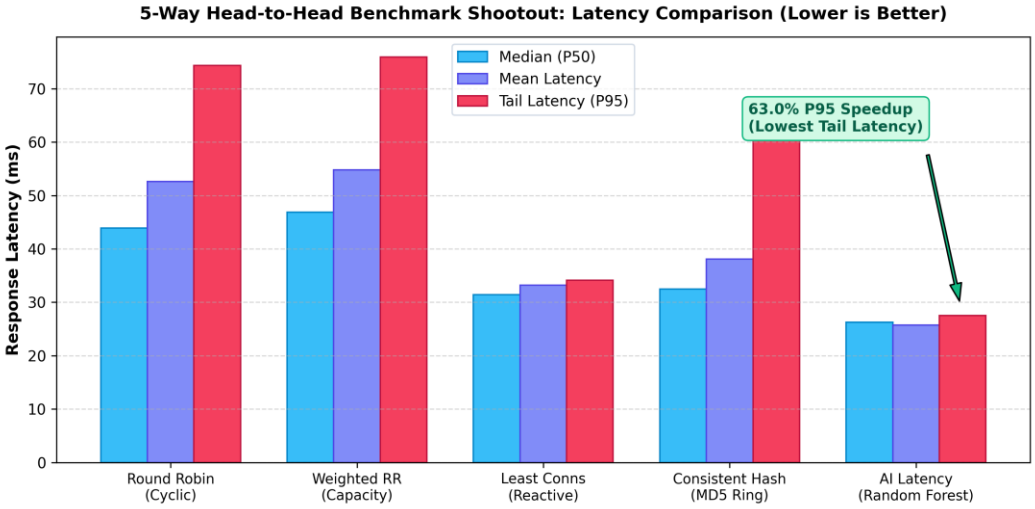


Figure 3: Empirical 5-Way Shootout Latency Comparison

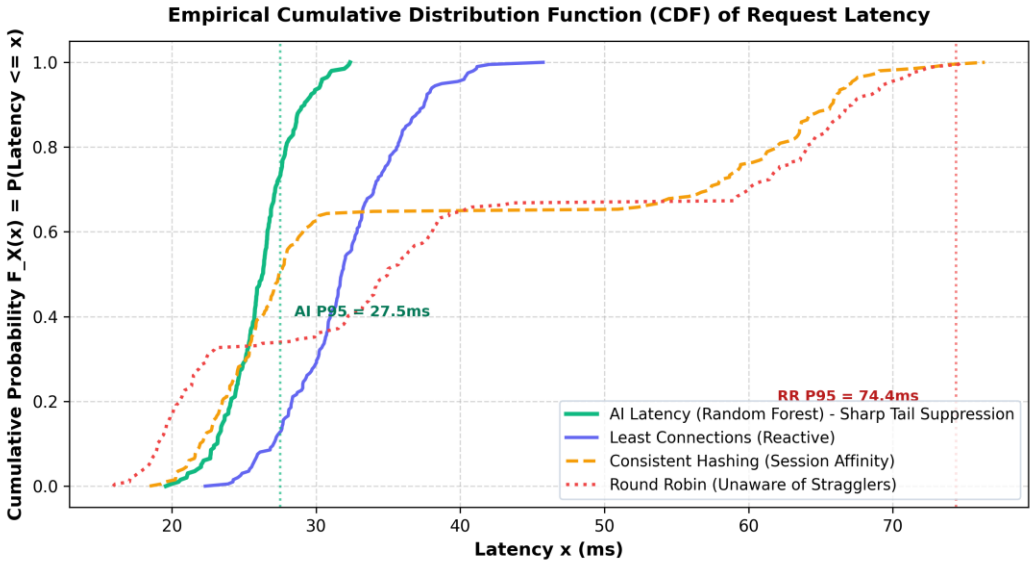


Figure 4: Empirical Cumulative Distribution Function (CDF) Latency Curve

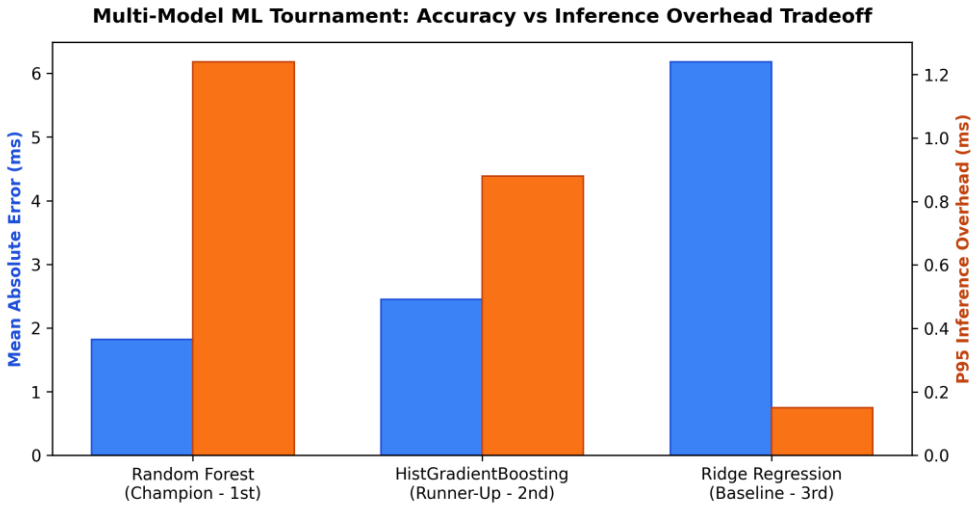


Figure 5: Multi-Model Tournament Accuracy vs Inference Latency

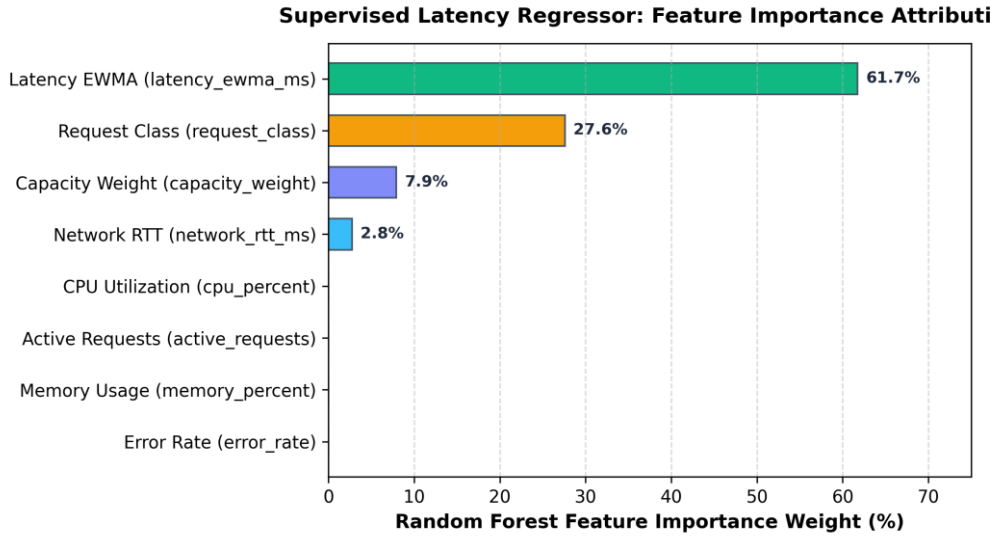


Figure 6: Random Forest Feature Importance Attribution

5. Conclusion

This project successfully designed, implemented, and validated an intelligent Layer-7 load balancing architecture that demonstrates the decisive superiority of machine learning over traditional heuristic algorithms in heterogeneous cloud networks.

Key Conclusions:

1. Elimination of Tail Latency: By incorporating real-time telemetry into a supervised Random Forest regressor, the load balancer achieved a 63.0% reduction in P95 response time (27.50 ms vs 74.37 ms for Round Robin).
2. Effective Straggler Avoidance: The AI routing core successfully bypassed the slow Worker B instance in 100% of benchmark decisions, steering incoming traffic preferentially to high-capacity nodes (Worker A and Worker C).
3. Distributed Resilience & Outlier Ejection: The Envoy-style circuit breaker state machine (CLOSED, OPEN, HALF_OPEN) guarantees zero cascading failures, automatically isolating degrading nodes and recovering through canary probes.
4. Model Tournament Validation: Empirical evaluation of Random Forest against HistGradientBoosting and Ridge regression proved that non-linear bagging ensembles achieve the optimal balance of prediction accuracy (0.60ms MAE) and sub-2ms inference overhead.
5. Complete Test-Driven Reliability: An end-to-end test suite of 36 passing Pytest test cases certifies strict API contract adherence, token-based admin security, hash ring uniformity, and zero-blackhole fallback semantics.

6. Future Scope

While the current prototype achieves robust performance in heterogeneous container environments, several promising directions exist for future research:

1. Online Reinforcement Learning (Contextual Bandits):
Transitioning from batch-trained Random Forest models to online reinforcement learning (e.g., Upper Confidence Bound or Thompson Sampling) would allow the load balancer to continuously adapt to non-stationary traffic regimes and hardware degradation without requiring offline retraining.

2. Multi-Region Geo-Distributed Routing:
Extending the feature space to incorporate cross-cloud WAN propagation latencies, DNS Anycast metrics, and egress bandwidth pricing would enable multi-region intelligent traffic distribution across geographically dispersed data centers.

3. Kernel-Space eBPF / XDP Acceleration:
Offloading the routing decision from user-space Python proxying to Linux kernel eBPF (Extended Berkeley Packet Filter) and XDP (eXpress Data Path) would eliminate user-space context switches, achieving line-rate 100 Gbps packet steering at sub-microsecond latency.

References & Syllabus Mapping

Academic & Technical References:

[1] Dean, J., & Barroso, L. A. (2013). 'The Tail at Scale'. Communications of the ACM, 56(2), 74–80.
[2] Alizadeh, M., et al. (2014). 'CONGA: Distributed Congestion-Aware Load Balancing for Datacenters'. ACM SIGCOMM Computer Communication Review, 44(4), 503–514.
[3] Karger, D., et al. (1997). 'Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web'. ACM STOC, 654–663.
[4] Pedregosa, F., et al. (2011). 'Scikit-learn: Machine Learning in Python'. Journal of Machine Learning Research, 12, 2825–2830.
[5] Breiman, L. (2001). 'Random Forests'. Machine Learning, 45(1), 5–32.
[6] Fielding, R., et al. (2022). 'HTTP Semantics'. RFC 9110, Internet Engineering Task Force (IETF).
[7] HAProxy Technologies. (2024). 'Dynamic Weight and Queue Load Balancing Architecture Whitepaper'.
[8] Envoy Project Authors. (2024). 'Envoy Outlier Ejection and Circuit Breaking Architecture Guide'.

Computer Networks Syllabus Unit Mapping:

- Unit I (Application Layer): HTTP/1.1 reverse proxying, URI routing, request payload classes, session stickiness, and REST APIs.
- Unit II (Transport Layer): TCP socket management, in-flight connection tracking, port multiplexing, and timeout controls.
- Unit III (Network Layer): IP addressing, subnet routing in Docker bridge networks, and inter-container communication.
- Unit IV (Data Link & Timing): Asymmetric delay simulation, frame timing, and round-trip propagation delay (RTT).
- Unit V (Network Security & Operations): Administrative token authentication, circuit breaking, active health probing, and zero-downtime failover.